“Operating System Design and Implementation”

(Chapter 5)
Security and Isolations in Operating Systems

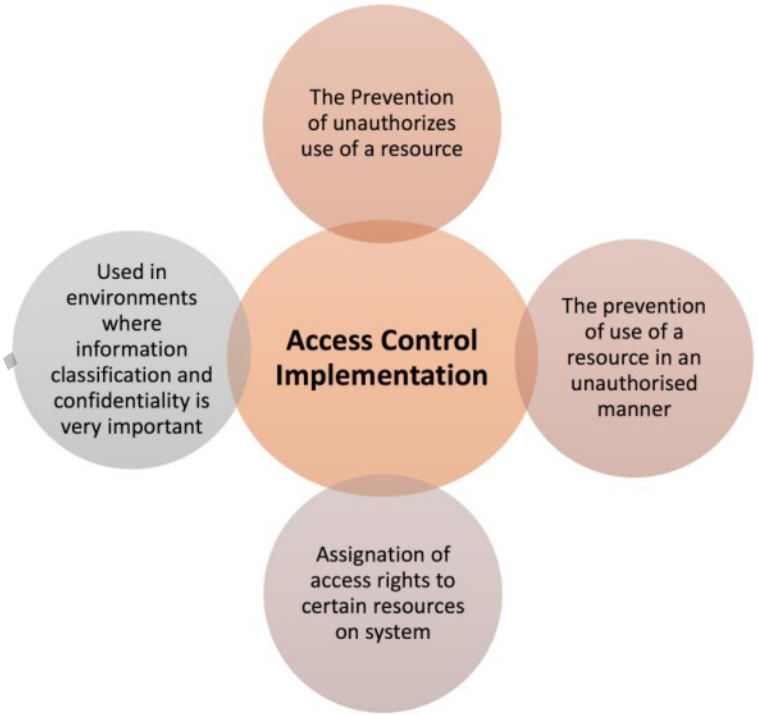
Contents

This Chapter will cover:

- Kernel Security Mechanisms: SELinux, AppArmor.
- Mitigation Techniques for Side-channel Attacks.
- Isolation Techniques: Namespaces, Cgroups, and Sandboxing.
- Address Space Layout Randomization (ASLR) and Stack Protection.
- Case Studies: Linux Security Module, Android Security Architecture, macOS Security Architecture)

Kernel Security Mechanisms

- Access control is a fundamental component of information security, dictating who can access certain resources and under what conditions.
- Ensures data integrity, confidentiality, and minimizes unauthorized access risks, thereby reducing the risk of data breaches and unauthorized data manipulation.
- Types
  1. Discretionary Access Control (DAC)
  2. Mandatory Access Control (MAC)
  3. Role-Based Access Control (RBAC)

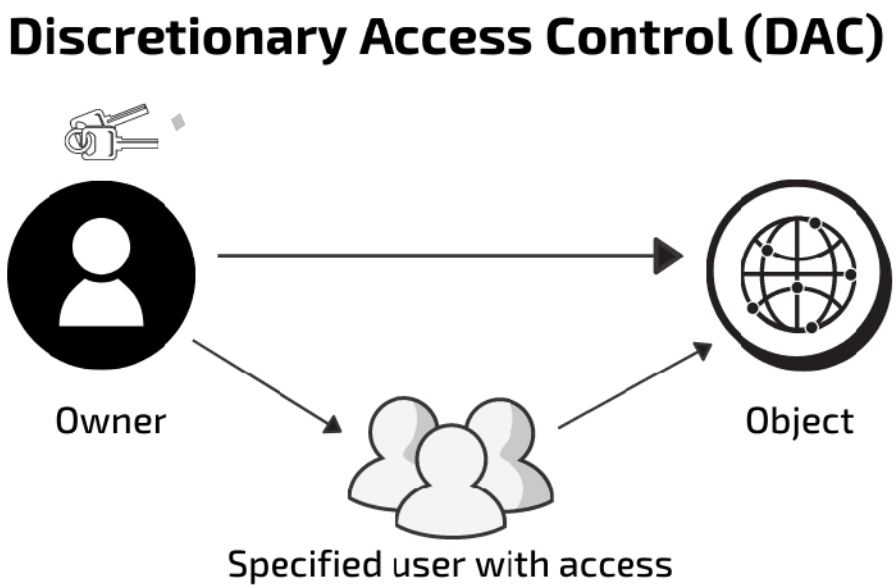


Kernel Security Mechanisms

Discretionary Access Control (DAC) Access

- DAC models allow the data owner to decide access control by assigning access rights to rules that users specify.
- When a user is granted access to a system, they can then set access permissions for other users.
- In a DAC model, security decisions are based solely on user identity and ownership of the objects.
- Prior to Linux kernel 2.6, DAC was the only security framework for Linux.
- Traditional Linux security mechanism ba group ownership
- Each user has complete discretion over his/her o
- Example:

In a file-sharing system, a document's owner can decide who can read, write, or execute the document. For instance, in Windows, a file owner can assign read-only access to a specific user while giving others full control over the file.

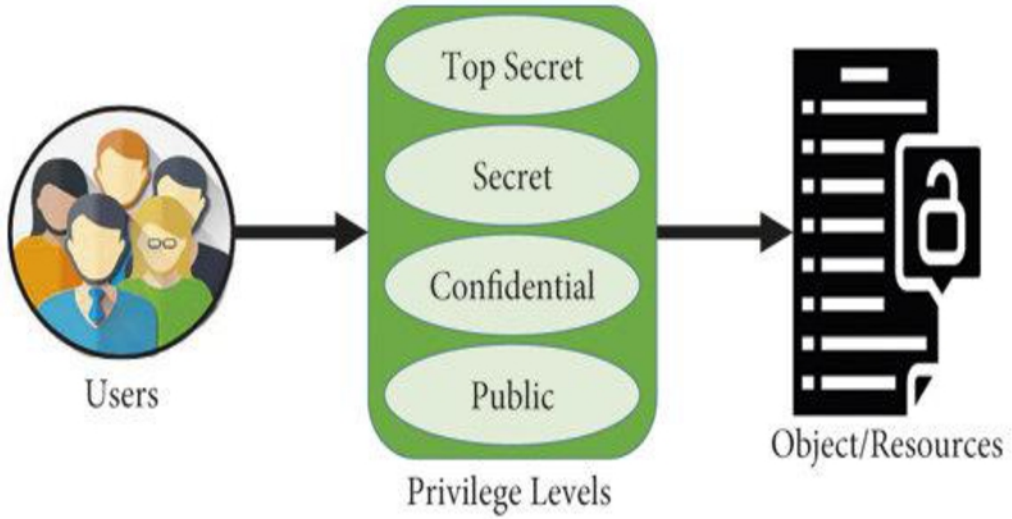


Kernel Security Mechanisms

Mandatory Access Control (MAC) Access

- MAC places rigid and strict access control policies on individual users and the data, resources, and systems they want to access.
- The policies are managed by central authority or an organization's administrator.
- Users are not able to alter, revoke, or set permissions.
- Example:

In a government system handling classified information, files are labeled as "Confidential," "Secret," or "Top Secret." A user with "Confidential" clearance cannot access "Top Secret" files unless they have the appropriate clearance level.

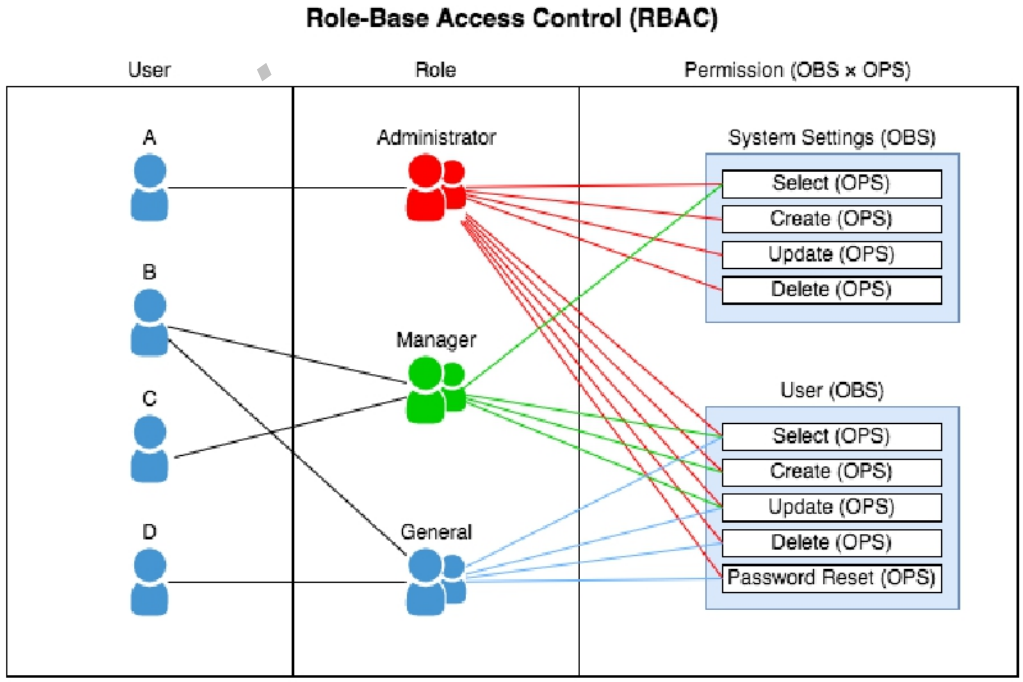


Kernel Security Mechanisms

Role-Based Access Control (RBAC) Access

- RBAC creates permissions based on groups of users, roles that users hold, and actions that users take.
- Users are able to perform any action enabled to their role and cannot change the access control level they are assigned.
- Similar to MAC, functions on access controls set by an authority, rather than by the owner of the resource.
- The difference between RBAC and MAC is that access control in RBAC is based on the role of the individual accessing the resource.
- Example:

In a hospital, doctors can access medical records, while administrative staff access billing information. Roles like "Doctor" and "Administrator" define access based on job functions, simplifying security management.



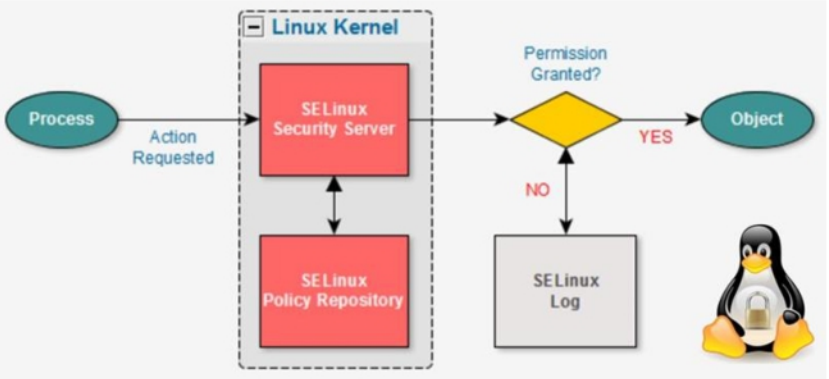
Kernel Security Mechanisms

Feature	Discretionary Access Control (DAC)	Mandatory Access Control (MAC)	Role-Based Access Control (RBAC)
Definition	Access is controlled by the resource owner	Access is controlled by a central authority	Access is granted based on a user's role within an organization
Flexibility	Highly flexible; users can change permissions	Very rigid; users cannot change permissions	Moderate; admins assign access to roles, not individual users
Use Case	Common in personal or less secure environments	Common in high-security environments (e.g., government)	Common in business environments with structured roles
Permission Control	Managed by the resource owner	Managed by a centralized security policy	Managed through assigned roles and permissions
Granularity	Permissions assigned at the user or group level	Access based on strict labels and levels	Access defined by role, not individual user
Example	File owner assigns read/write access in a file-sharing system	Military systems restrict classified information based on security clearance	Doctors can access medical records; administrators access billing records
Risk of Unauthorized Access	Higher, as users control permissions	Lower, as access is tightly controlled by policies	Moderate, as access depends on role setup and assignment

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux-Security Enhanced Linux

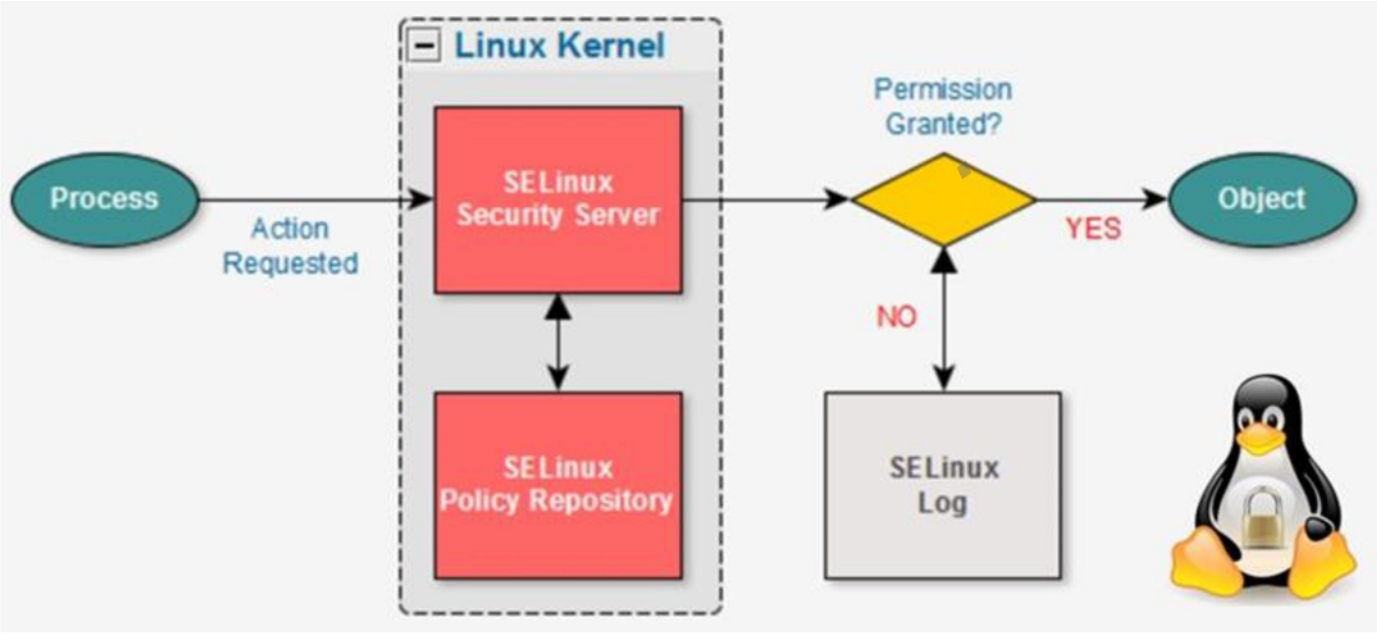
- SELinux stands for Security-Enhanced Linux, and it's a security system built into Linux to control how programs and users can access files, processes, and other resources on the system.
- It was originally developed by the United States National Security Agency (NSA) as a mechanism to enhance security and restrict unauthorized access to system resources.
- Unlike traditional Linux security mechanisms(DAC based), which rely on file permissions and user roles, SELinux is a MAC security framework integrated into the Linux kernel in which access is granted based on security labels rather than ownership.



Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux-Security Enhanced Linux

- Instead of allowing access based on who owns the file, in SELinux, every file, process, or system resource gets a label. This label is called a security context. It's like a "tag" that contains information about what that file or process is, and who is allowed to interact with it.



Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux-Security Enhanced Linux

- In SELinux, these security labels are part of a Security Context, which consists of:
  - ✓ User (user_u): SELinux user identity . This part identifies the SELinux user. It's like the identity of the person or process that's interacting with the system. It's used by SELinux to control access. Example: user_u (for a regular user), sysadm_u (for a system administrator).
  - ✓ Role (role_r): Defines what actions the user can perform. This part defines what actions the user or process can perform. It's like saying, "If you are this kind of user or process, you can do these specific things, but not others." Example: role_r means it's a general role; sysadm_r could mean it's a system administrator role with more permissions.
  - Type (type_t): The most important attribute, defining what the resource is(object) and who can access it(object).This part is the most important for controlling what the resource is (like a file, directory, or process) and who can access it. It's like saying, "Only this type of process can access files of this type." Example: httpd_t for web server processes, user_home_t for personal user directories, httpd_sys_content_t for web content files.

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux-Security Enhanced Linux

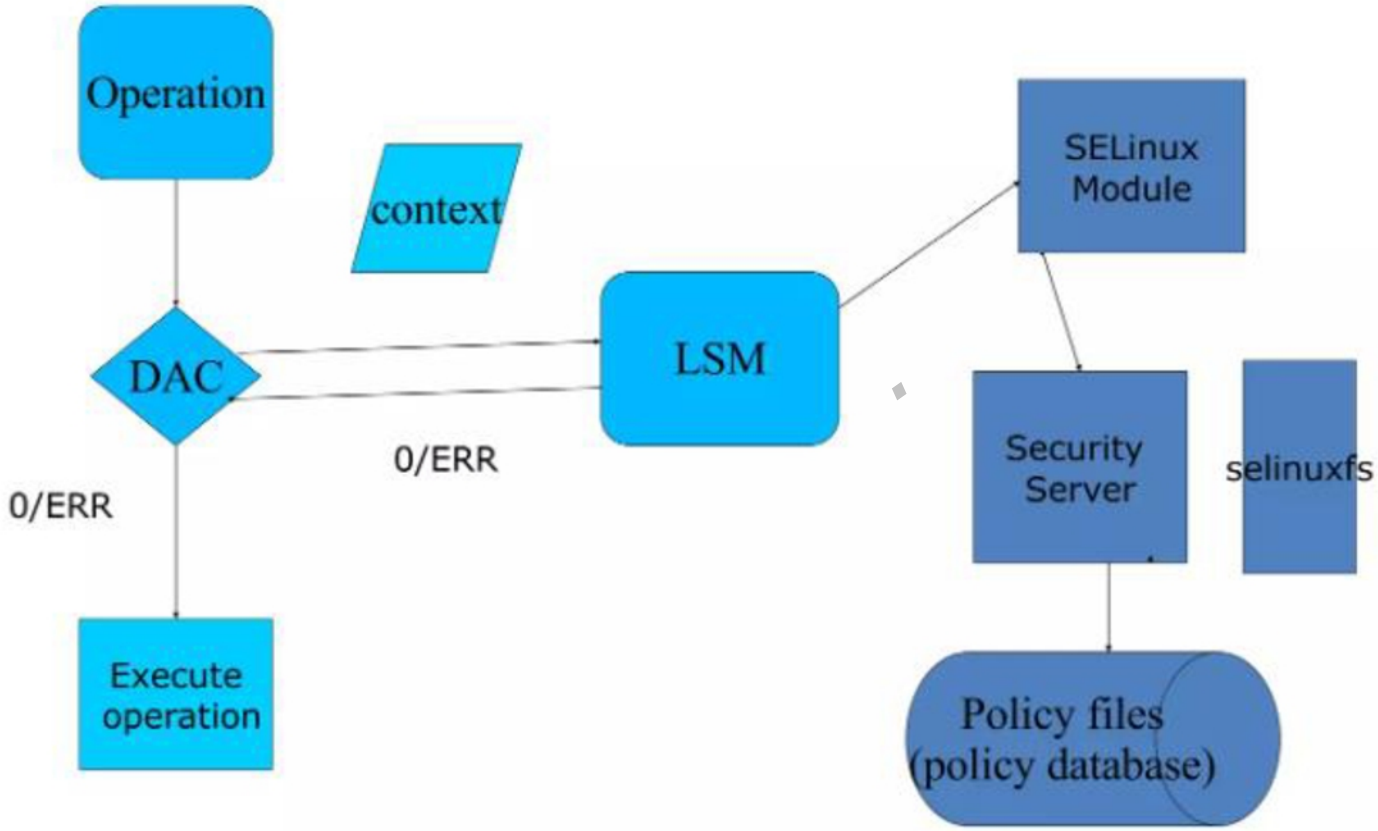
- Next, SELinux works by using policies. The Policies define what is allowed based on these labels. Security policies are defined centrally and cannot be overridden by users. These policies are like rules that say, "Only certain processes with certain labels can access certain files or resources." The rules are set by system administrators and cannot be easily changed by regular users.

Example:

- Imagine you have a web server (Apache) running on your Linux machine. It needs to read files from the web content folder (/var/www/html/). The web server has a security context like httpd_t, which is allowed to access files with the context httpd_sys_content_t (web files). If the web server tries to access your home directory (/home/user/), where files are labeled user_home_t, SELinux will block it because httpd_t (web server process) isn't allowed to access files with the user_home_t label as defined in the policy database.

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux Architecture



Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux Architecture
SELinux consists of several key components that define how access control is implemented:

Operation

- This represents any action performed on the system, such as opening a file, executing a program, or accessing a resource.
- The request must pass through security checks before execution.

DAC

- Traditional Linux security mechanism based on file permissions (rwx), user ownership, and group ownership.
- The first security layer in Linux:
- If DAC allows the operation → it moves to the LSM layer.
- If DAC denies the operation → it is blocked immediately.

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux Architecture
SELinux consists of several key components that define how access control is implemented:

LSM (Linux Security Modules)

- LSM is a framework that allows different security models (like SELinux, AppArmor to be plugged into the Linux kernel.
- Acts as a gateway for additional security checks after DAC.
- If DAC approves, LSM then checks if SELinux policies allow or deny the request.
- The SELinux kernel module is part of the Linux Security Module (LSM) framework.
- It enforces Mandatory Access Control (MAC) policies by interacting with LSM.
- Controls what operations a process or user can perform, based on security contexts and policies.

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux Architecture
SELinux consists of several key components that define how access control is implemented:

Security Server

- The Security Server acts as the decision-making component of SELinux.
- It:
  - ✓ Receives access requests from LSM.
  - ✓ Consults policy files (stored in the policy database).
  - ✓ Determines whether to allow or deny the operation.

Policy Files (Policy Database)

- A database of security policies that define access control rules.
- Contains rules specifying which processes can access which resources.

Kernel Security Mechanisms

Kernel Security Mechanisms: SELinux Architecture
SELinux consists of several key components that define how access control is implemented:

selinuxfs (SELinux Filesystem)

- A virtual filesystem (/sys/fs/selinux) that provides an interface for managing SELinux policies and status

Execution of Operation

- If SELinux grants permission, the operation is executed.
- If SELinux denies it, the system logs the denial (in /var/log/audit/audit.log).

Side-channel Attacks

Introduction

- Side-channel attacks (SCAs) exploit unintended leakage of information from cryptographic and computational systems.
- In computer security, a side-channel attack is any attack based on extra information that can be gathered because of the fundamental way a computer algorithm is implemented, rather than flaws in the design of the algorithm itself, but potentially devastating, mistakes or oversights in the implementations.
- Unlike direct attacks on an algorithm's mathematical structure, SCAs derive sensitive data by analyzing physical characteristics such as power consumption, electromagnetic emissions, timing variations, and acoustic signals.
- These attacks pose a serious threat to secure systems, especially in embedded devices, smart cards, and secure processors.

Side-channel Attacks

General Classes of Side channel attack

- **Cache attack:** attacks based on attacker's ability to monitor cache accesses made by the victim in a shared physical system as in virtualized environment or a type of cloud service.
- **Timing attack:** attacks based on measuring how much time various computations (such as, say, comparing an attacker's given password with the victim's unknown one) take to perform.
- **Power monitoring attack:** attacks that make use of varying power consumption by the hardware during computation.
- **Electromagnetic attack:** attacks based on leaked electromagnetic radiation, which can directly provide plaintexts and other information. Such measurements can be used to infer cryptographic keys using techniques equivalent to those in power analysis or can be used in non-cryptographic attacks
- **Acoustic cryptoanalysis:** attacks that exploit sound produced during a computation (rather like power analysis).
- **Data remanence:** in which sensitive data are read after supposedly having been deleted. (e.g. Cold boot attack). Data remanence refers to residual traces of data that remain on a storage medium even after the data has been deleted or erased
- **Whitelist:** attacks based on the fact that the whitelisting devices will behave differently when communicating with whitelisted (sending back the responses) and non-whitelisted (not responding to the devices at all) devices. Whitelist-based side channel may be used to track Bluetooth MAC addresses.

Side-channel Attacks

Mitigating Side channel attack

- Side channels can hard to eliminate entirely, but there are several best practices that can be used to mitigate common side channels.
- Because side-channel attacks rely on the relationship between information emitted (leaked) through a side channel and the secret data, countermeasures fall into two main categories:
  - Eliminate or reduce the release of such information
  - Eliminate the relationship between the leaked information and the secret data, that is, make the leaked information unrelated, or rather *uncorrelated*, to the secret data, typically through some form of randomization of the ciphertext that transforms the data in a way that can be undone after the cryptographic operation (e.g., decryption) is completed.

Example: Constant Time Programming

The idea behind constant-time programming is simple: eliminate timing variations by writing a program that always takes the same amount of time to run. However, this is much harder than it might seem at rst glance, for a number of reasons.

Real Time System

Further Study

- DAC vs MAC
- SELinux Vs AppArmor
- Write short notes on
    1. Isolation Techniques : Namespaces, Cgroups, and Sandboxing
    2. Address Space Layout Randomization (ASLR) and Stack Protection
- Case Studies: Linux Security Module, Android Security Architecture, macOS Security Architecture)

Real Time System

Further Study

```
            System.out.print("Address: "+rowSet.getString("Address")+", ");
            System.out.println("");
        }

    //Inserting data into RowSet object

    rowSet.moveToInsertRow();
    rowSet.updateInt(1, 6001);
    rowSet.updateString(2, "Ram Thapa");
    rowSet.updateString(3, "Pokhara");
    rowSet.insertRow();
    System.out.println("One Row Inserted Successfully");

  }
}
```

Assignment

1. **Briefly explain Row Sets in java with suitable programming example.**